05

MGU BCA - 3RD SEM

Database Management Systems

Module 5 - Practical

By Aben Sojan

Exam - 6 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

MGU BCA Honours DBMS (MG3CCRBCA201): 70 marks, 2 hours. Part A: 5 x 2 marks, short questions from all modules. Part B: answer any 5 of 6-mark questions. Part C: answer any 2 of 15-mark essays. Old CBCS papers (2019-2021): 80 marks, 3 hours, any-10 x 2 + any-6 x 5 + any-2 x 15. Part C hotspots: COMPANY ER diagram, Normal Forms essay, anomalies.

TOPIC 1

ER Diagram Practice

1

Step 1: list the entities, e.g. Student, Course, Teacher.

Step 2: list attributes for each and underline the key attributes.

Step 3: find relationships, e.g. Student enrolls in Course, Teacher teaches Course.

Step 4: draw rectangles, ellipses, diamonds and connect them with lines.

Step 5: mark weak entities with double rectangles if any exist.

QUESTION BANK

1 Draw an ER diagram for a college database with Student, Course and Teacher. Describe it.

IMPORTANT

This practical task is to draw an ER diagram for a college database with three entities: Student, Course and Teacher.

Entities with their attributes (key attributes are underlined in the diagram):

```
Student(rollno, name) where rollno is the primary key.
Course(code, title) where code is the primary key.
Teacher(id, name) where id is the primary key.
```

Relationships: Student enrolls in Course (many-to-many); Teacher teaches Course (one-to-many).

Draw rectangles for the three entities Student, Course and Teacher.

Draw diamonds for the relationships Enrolls and Teaches, and connect each diamond to its two entity rectangles with lines.

Draw ellipses for attributes and connect them to the entity rectangles; underline the key attribute ellipse of each entity.

Many-to-many Enrolls would become its own table with rollno and code as foreign keys, while the one-to-many Teaches is handled by a foreign key of Teacher in the Course table.

2 How do you convert a many-to-many relationship into tables?

IMPORTANT

A many-to-many relationship cannot be stored in either entity table directly, so it is converted into its own table.

Create a separate table for the relationship itself.

```
CREATE TABLE Enrolls (
rollno INT,
code VARCHAR(10),
grade CHAR(2),
PRIMARY KEY (rollno, code),
FOREIGN KEY (rollno) REFERENCES Student(rollno),
FOREIGN KEY (code) REFERENCES Course(code)
);
```

Example: Enrolls(rollno, code) with both columns as foreign keys.

The pair (rollno, code) together forms the composite primary key, so one student cannot be enrolled in the same course twice.

Extra attributes of the relationship, like grade, go in this table too.

This new table is called a junction table; it removes the many-to-many link and replaces it with two one-to-many links.

TOPIC 2

DDL and DML Practice

2

CREATE TABLE defines columns, keys and constraints in one shot.

INSERT adds rows; always list columns to stay safe.

UPDATE changes rows matched by WHERE; DELETE removes them.

Forgetting WHERE in UPDATE or DELETE changes every row, so double-check it.

QUESTION BANK

3 Write SQL to create an Employee table and insert three rows.

IMPORTANT

This practical task is to create an Employee table with constraints using DDL and then add three rows using DML.

Create the table:

```
CREATE TABLE Employee (
id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
salary INT CHECK (salary > 0)
);
```

Insert rows:

```
INSERT INTO Employee VALUES (1, 'Anu', 30000);
INSERT INTO Employee VALUES (2, 'Ben', 45000);
INSERT INTO Employee VALUES (3, 'Cia', 38000);
```

The PRIMARY KEY on id guarantees every employee row has a unique id.

NOT NULL on name stops blank names from being inserted, and CHECK (salary > 0) rejects zero or negative salaries.

Each INSERT adds one complete row; after the three statements the table holds 3 employees.

Expected output: the table Employee with columns id, name, salary and three rows for Anu, Ben and Cia.

4 Write queries to give Ben a 5000 raise and to delete the employee named Cia. (Mar 2021)

IMPORTANT

This practical task is to modify an existing Employee table: raise the salary of Ben and remove the row of Cia.

Raise salary with UPDATE and WHERE:

```
UPDATE Employee SET salary = salary + 5000 WHERE name = 'Ben';
```

Remove the row with DELETE and WHERE:

```
DELETE FROM Employee WHERE name = 'Cia';
```

The WHERE clause in the UPDATE limits the change to only the row where name is Ben, so other salaries stay untouched.

Forgetting WHERE in UPDATE or DELETE would change or remove every row, so always double-check the condition first.

Expected result: Ben's salary becomes 50000 and the table has only Anu and Ben left after Cia's row is deleted.

TOPIC 3

SQL Query Practice: Joins and Set Operators

3

Set operators UNION, INTERSECT and EXCEPT combine results of two queries.

Both queries must return the same number and type of columns.

UNION removes duplicates; UNION ALL keeps them.

Typical exam pattern: join two tables, filter with WHERE, group and filter groups with HAVING.

QUESTION BANK

5 **Explain UNION, INTERSECT and EXCEPT with one example each. (Dec 2021, Mar 2021)**

REPEATED

Set operators UNION, INTERSECT and EXCEPT combine the results of two queries into one result.

UNION: students from section A plus section B, duplicates removed.

```
SELECT name FROM SecA UNION SELECT name FROM SecB;
```

INTERSECT: names present in both sections.

```
SELECT name FROM SecA INTERSECT SELECT name FROM SecB;
```

EXCEPT: names in A but not in B.

```
SELECT name FROM SecA EXCEPT SELECT name FROM SecB;
```

Both queries must return the same number and type of columns, otherwise the operator gives an error.

UNION removes duplicate rows; UNION ALL keeps the duplicates.

Expected output: each query returns a single name column with the combined, common, or differing names of the two sections.

6 Write a query to list student names with their subject and mark using INNER JOIN.

IMPORTANT

This practical task is to combine data from two tables, Student and Marks, so that each student appears with his subject and mark.

Join on the common rollno column:

```
SELECT s.name, m.subject, m.mark
FROM Student s INNER JOIN Marks m
ON s.rollno = m.rollno;
```

INNER JOIN matches rows where the ON condition is true, so only students having marks rows appear in the result.

The aliases s and m are shortcuts for the Student and Marks tables, making the query shorter and easier to read.

Expected output: one row per student-mark pair showing name, subject and mark; a student with no marks row is left out.

7 Write a query to find departments with average marks above 70, sorted high to low.

IMPORTANT

This practical task is to find departments whose average marks are above 70 and list them from the highest average to the lowest.

Group by department, filter groups with HAVING, sort with ORDER BY:

```
SELECT dept, AVG(marks) AS avg_marks
FROM Student GROUP BY dept
HAVING AVG(marks) > 70
ORDER BY avg_marks DESC;
```

GROUP BY dept collects the students of each department into groups, and AVG(marks) computes the average of each group.

HAVING filters groups after grouping (WHERE filters single rows before grouping), keeping only departments with average above 70.

ORDER BY avg_marks DESC sorts the surviving departments from the highest average to the lowest.

Expected output: one row per qualifying department with its dept name and avg_marks, highest first.

TOPIC 4

4

PL/SQL Basics

PL/SQL adds programming to SQL: variables, conditions, loops and error handling.

Block structure: DECLARE, BEGIN, EXCEPTION, END.

DECLARE holds variables; BEGIN holds the logic; EXCEPTION handles errors.

Used for triggers, procedures and batch jobs inside the database.

QUESTION BANK

8 **What is PL/SQL? Show the structure of a PL/SQL block with an example.**

IMPORTANT

PL/SQL is Oracle procedural language that adds variables and logic to SQL.

It is used for triggers, procedures and batch jobs inside the database.

Basic block structure:

```
DECLARE
m NUMBER;
BEGIN
SELECT marks INTO m FROM Student WHERE rollno = 1;
DBMS_OUTPUT.PUT_LINE('Marks: ' || m);
EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('No student found');
END;
/
```

DECLARE holds variables; BEGIN holds the logic; EXCEPTION handles errors.

SELECT ... INTO m reads a value from the table into the variable m.

The EXCEPTION part catches the NO_DATA_FOUND error when no student has rollno 1 and prints a friendly message instead of crashing.

9 Write a PL/SQL block that prints "Pass" if marks are above 40, else "Fail".

IMPORTANT

This practical task is to write a PL/SQL block that prints "Pass" if marks are above 40, else prints "Fail".

Use an IF condition inside the BEGIN block:

```
DECLARE
m NUMBER := 55;
BEGIN
IF m > 40 THEN
DBMS_OUTPUT.PUT_LINE('Pass');
ELSE
DBMS_OUTPUT.PUT_LINE('Fail');
END IF;
END;
/
```

The variable m is given the value 55 in the DECLARE section, so the condition m > 40 is true.

IF ... THEN ... ELSE ... END IF is the PL/SQL way of choosing between two actions.

DBMS_OUTPUT.PUT_LINE prints the text to the output window.

Expected output: Pass, because 55 is above 40; with m = 30 the output would be Fail.

TOPIC 5

MongoDB Query Practice

5

use college switches to (or creates) a database.

db.createCollection("students") makes a collection.

insertMany adds several documents at once.

find with operators filters; updateMany and deleteMany act on many documents.

QUESTION BANK

10 Write MongoDB commands to create a students collection, insert two documents and find toppers.

IMPORTANT

This practical task is to build a small students database in MongoDB: create a collection, insert two documents, and find the toppers.

Switch database and create the collection:

```
use college;
db.createCollection("students");
```

Insert documents:

```
db.students.insertMany([
{name: "Aben", marks: 92},
{name: "Ben", marks: 74}
]);
```

Find students scoring above 85:

```
db.students.find({marks: {$gt: 85}});
```

use college switches to the college database and creates it if it does not exist.

insertMany adds both student documents in one call; MongoDB collections are flexible, so no fixed schema is needed.

The $gt operator means greater than, so find returns only documents with marks above 85.

Expected output: one document for Aben with marks 92; Ben with marks 74 is left out of the result.

11 Write MongoDB commands to update marks and delete a student.

IMPORTANT

This practical task is to update several documents at once and then delete one specific document in MongoDB.

Update matching documents:

```
db.students.updateMany({marks: {$lt: 40}}, {$set: {result: "fail"}});
```

Delete one document:

```
db.students.deleteOne({name: "Ben"});
```

updateMany finds every document whose marks are below 40 ($lt means less than) and adds a result field with value "fail" using $set.

The $set operator changes only the listed fields and leaves the rest of the document untouched.

deleteOne removes the first document matching the filter, here the student named Ben.

Expected result: all students with marks below 40 get result: "fail", and Ben's document is gone from the collection.